

ChuvaApp — Documento de Arquitetura

Visão Geral

Aplicativo web (PWA) que calcula a **rota mais segura entre dois pontos** considerando zonas de alagamento previstas para os **próximos 30 a 60 minutos**. O app cruza dados de previsão de chuva em tempo real com mapas históricos de risco de inundação das regiões de São Paulo e Guarulhos, entregando ao usuário a rota antes que o problema aconteça.

O diferencial frente a apps como RainViewer é a pergunta que o ChuvaApp responde: *"o meu caminho vai alagar antes de eu chegar?"* — não apenas onde está chovendo agora.

Requisitos Funcionais

- Usuário informa origem e destino e recebe a rota mais segura
- A rota considera zonas de risco de alagamento + previsão de chuva para os próximos 30–60 min
- Mapa exibe sobreposição visual das zonas de intensidade de chuva
- Cadastro com login — salva histórico de pesquisas recentes
- Alertas quando uma rota salva está em risco de alagamento

Planilha de Características Arquiteturais

7 Características Explícitas

#	Característica	Origem nos Requisitos
1	Availability	App não pode cair durante chuvas intensas — justamente quando é mais necessário
2	Fault Tolerance	Fallback quando APIs externas bloquearem por rate limit
3	Reliability	Dado correto vale mais que dado rápido — rota errada = usuário preso
4	Elasticity	Pico de acesso durante chuvas + controle assíncrono de chamadas às APIs
5	Data Freshness	Dados de radar com ciclo de atualização de 10 minutos
6	Personalization	Login com histórico de pesquisas recentes por usuário
7	Responsiveness	Rota calculada antes do usuário sair de casa

Top 3 — Características Primárias

Posição	Característica	Justificativa
1º	Availability	O app precisa estar no ar no pior momento — durante a chuva intensa
2º	Fault Tolerance	Toda a proposta de valor depende de APIs externas; se elas falharem, o sistema não pode cair junto
3º	Reliability	Dado desatualizado é mais perigoso que dado lento — usuário pode tomar rota que já alagou

Características Implícitas

Característica	Justificativa
Security	Sistema com cadastro e histórico de pesquisas por usuário
Maintainability	Solo developer — arquitetura precisa ser simples de evoluir sem time
Observability	Necessário detectar quando o radar parou de atualizar ou API foi bloqueada antes que o usuário perceba

Outras Considerações Arquiteturais

1. PACELC → Perfil PA/EL (sistema como um todo)

O ChuvaApp adota o modelo PACELC por ser mais preciso que o CAP para sistemas de dados em tempo real. O CAP só considera cenários de falha de partição; o PACELC também pergunta: em operação normal, o sistema prefere **Latência baixa** ou **Consistência**?

- **P → A (Availability):** Em caso de partição de rede, o sistema responde com dado em cache levemente desatualizado ao invés de retornar erro
- **E → L (Latency):** Em operação normal, prefere responder rápido com dado do Redis a aguardar confirmação da API externa

Dado de 10 minutos atrás ainda é útil para evitar um alagamento. Essa é uma decisão consciente — não um descuido.

2. Dependência de Terceiros é o Maior Risco

Toda a proposta de valor depende de APIs externas:

- Previsão de chuva: OpenWeatherMap / Open-Meteo (fallback)
- Zonas de risco: IPT / GeoSampa SP
- Roteamento: OpenRouteService

Se todas falharem simultaneamente, o app perde o core. O **Fault Tolerance** no Top 3 existe exatamente por isso. Mitigação: fallback em cascata e Redis como fonte de verdade com TTL de 10 minutos.

3. Restrição de Time

Solo developer por enquanto. Essa restrição:

- Justifica as implícitas de **Maintainability** e **Observability**
- Limita deliberadamente a complexidade arquitetural no MVP
- Favorece soluções com dados públicos prontos (IPT, GeoSampa) ao invés de construir modelos próprios

Estilo Arquitetural

Decisão: Microserviços + Event-Driven (Kafka)

Escolhido por ser o padrão dominante em bancos digitais e financeiras no Brasil (Nubank, Inter, C6, Santander modernização). Demonstra domínio do stack esperado em entrevistas técnicas para posições backend Java.

Stack Tecnológico

Camada	Tecnologia
API Gateway	Spring Cloud Gateway
Microserviços	Spring Boot 3 + Java 21
Mensageria async	Apache Kafka
Cache	Redis
Autenticação	Spring Security + JWT
Observability	Spring Actuator + Prometheus + Grafana + Zipkin + ELK
Deploy	Docker + Docker Compose

Os 4 Microserviços

Bounded Contexts

Serviço	Responsabilidade
Weather Service	Busca previsão de chuva dos próximos 30–60 min nas APIs externas. Atualiza o cache de radar. Publica evento radar . updated a cada 10 minutos
Routing Service	Recebe origem + destino. Calcula rota mais segura cruzando zonas de risco com previsão atual. Publica evento route . calculated
User Service	Cadastro, login (JWT), histórico de pesquisas, rotas salvas. Publica evento route . saved quando usuário salva uma rota

Serviço	Responsabilidade
Alert Service	Não expõe API REST. Consome eventos radar . updated e route . saved do Kafka. Cruza os dois e dispara notificação se uma rota salva está em zona de risco

Infraestrutura de Suporte

Componente	Papel
API Gateway	Entrada única. Valida JWT, roteia para o serviço correto. Stateless — sem banco de dados
Observability Stack	Prometheus + Grafana (métricas), Zipkin + Spring Sleuth (tracing distribuído), ELK (logs)

Fluxo de Eventos Kafka

```

Weather Service  → [radar.updated]      → Alert Service → notifica usuário
User Service     → [route.saved]        → Alert Service → notifica usuário
Routing Service  → [route.calculated]    → User Service  → salva no histórico

```

Padrão de Comunicação

- **Síncrono (REST):** Cliente → API Gateway → Weather / Routing / User Service
- **Assíncrono (Kafka):** Serviços publicam eventos → Alert Service reage
- **Push para cliente (SSE/WebSocket):** Alert Service → API Gateway → cliente

Decisão de Banco por Serviço (PACELC)

Cada serviço tem seu próprio perfil PACELC baseado no **impacto da falha**. O Routing Service é o único CP porque uma rota incorreta coloca o usuário em risco real. Os demais preferem disponibilidade — um alerta com 2 minutos de atraso é melhor que nenhum alerta.

Serviço	PACELC	Banco	Justificativa
Weather Service	PA/EL	Redis	Dado de radar eventual é aceitável — velocidade e disponibilidade são prioridade
Routing Service	PC/EC	CockroachDB (Raft)	Rota com dado errado de zona de alagamento é perigosa — consistência obrigatória
User Service	PA/EL	DynamoDB	Histórico e login toleram consistência eventual — salvar rota 200ms depois não impacta o usuário
Alert Service	PA/EL	Redis	Disponível durante chuva intensa é mais crítico que consistência perfeita
API Gateway	—	Stateless	Sem banco — JWT em memória, sem estado persistido
Observability	PA/EL	DynamoDB	Logs e métricas não precisam de consistência forte

Nota sobre Raft: Raft é um algoritmo de consenso, não um banco de dados. O banco que o implementa é o CockroachDB — que garante consistência distribuída em múltiplos nós com failover automático.

Observability Stack

Em microserviços, quando algo falha não é óbvio em qual serviço o problema ocorreu. O stack de observability cobre os 3 pilares:

Pilar	Ferramenta	Pergunta que responde
Métricas	Prometheus + Grafana	"O Weather Service está com latência alta?"
Tracing	Zipkin + Spring Sleuth	"Em qual serviço a requisição travou?"
Logs	ELK Stack (Elasticsearch + Logstash + Kibana)	"O que exatamente aconteceu naquele momento?"

Todos os 4 microserviços expõem métricas e traces automaticamente via Spring Actuator + Spring Sleuth. O Alert Service é monitorado com especial atenção — silêncio durante chuva intensa é um sinal de alerta crítico.

Contexto Adicional

- **Dados públicos disponíveis:** IPT tem carta de suscetibilidade a inundações de Guarulhos via WMS; GeoSampa SP tem histórico de ocorrências de alagamento com API WFS/WMS aberta
- **Janela de 30–60 min é intencional:** Previsão acima de 1h perde confiabilidade — o app não promete o que não pode entregar
- **PWA first:** Funciona no browser, instalável no celular sem loja. Backend não muda quando evoluir para mobile nativo
- **Rate limiting é tratado pelo Fault Tolerance:** Weather Service implementa fallback automático de OpenWeatherMap para Open-Meteo quando rate limit é atingido, sem expor o erro ao usuário